

# Introdução à Computação achando erros

Alexandre Rademaker

# Depuração (debug) I

Bugs são erros que fazem com que o programa se comporte de forma diferente do pretendido. E a depuração é o processo de encontrar e corrigir esses bugs.

Uma forma de **debug** é imprimir valores das variáveis em determinados pontos do programa.

Uma boa técnica de depuração é explicar nosso próprio código em voz alta para alguém (ou alguma coisa!).

Em cada ambiente o suporte pode ser diferente. Vamos experimentar no **Codespaces**!

# Depuração (debug) II

Queremos 3 blocos na tela. Qual o bug?

buggy0.c

```
1 #include <stdio.h>
2
3 int main(void) {
4     for (int i = 0; i <= 3; i++)
5         printf("#\n");
6
7 }
```

# Depuração (debug) III

Qual o bug? No VS Code, usar 'step into'.

buggy1.c

```
1  #include <stdio.h>
2
3  int get_negative_int(void);
4
5  int main(void)
6  {
7      int i = get_negative_int();
8      printf("%i\n", i);
9  }
10
11 int get_negative_int(void) {
12     int n;
13     do {
14         printf("Negative Integer: ");
15         scanf("%d", &n);
16     } while (n < 0);
17     printf("The value is %d", n);
18     return 0;
19 }
```

## Depuração (debug) IV

Lembrando do problema em `src/mario2.c` . Vamos adicionar um pouco de robustez em `mario3.c` .

Queremos garantir que a entrada passada pelo usuário é compatível com o que esperamos.

Usamos o `printf` para entender os valores.

Precisamos de algumas bibliotecas adicionais para poder usar o valor `true` e a função `isdigit` .

# Depuração (debug) V

mario3.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     // input
6     int n;
7     do {
8         printf("size: "); scanf("%d", &n);
9     } while (n < 1);
10
11     // output
12     for (int i = 0; i < n; i++) {
13         for (int j = 0; j < n; j++)
14             printf("#");
15         printf("\n");
16     }
17 }
```